

Day 4 — Anticipated Questions and Model Answers

Doubt clarification for computer vision, MLOps,
active learning and feedback loops

Questions participants are likely to raise, with answers written the way you would say them in the room. Difficult and challenging questions are marked, with guidance on handling them without losing the room or overclaiming.

Enterprise AI Engineering Bootcamp · Companion to the Day 4 presentation and study material

A. Computer Vision Fundamentals

Q How many images do we need to train a defect detection model?

The wrong question, and I say that gently because everyone asks it. The right question is which constraint you are actually hitting. If your labels are inconsistent, ten thousand images will not help — they will add ten thousand inconsistencies. If the defect is smaller than your optics can resolve, no volume of images contains the information. If a specific class has twelve examples, then yes, targeted collection of that class will help substantially. Run the diagnosis in Lab 1 first; the answer changes completely depending on the row you are in.

Q Detection or segmentation?

Detection if you need to know what and where. Segmentation if extent or area matters — corrosion coverage, weld length, the percentage of a surface affected. Segmentation labelling costs several times more per image, so only choose it when a measurement of extent actually drives a decision. Teams frequently choose segmentation because it looks more sophisticated and then discover the labelling budget was spent producing information nobody uses.

Q We have very few defect examples. Is CV viable at all?

Yes, but probably not as supervised classification. Anomaly detection trains on images of good parts only and flags anything unusual — which is exactly the situation you are describing, and it is a fundamentally easier data problem. You then use your handful of real defects to validate that the anomaly score rises on them. That is a much more achievable claim than training a classifier on twelve positives, and it is the same reformulation we applied to predictive maintenance on Day 2.

Q Which model architecture should we use?

It matters less than you expect, and it is not the first decision. Establish the inference location and latency budget, the defect size in pixels, and the licence position — those three constrain the choice more than any accuracy consideration. Then pick something well-supported rather than leaderboard-topping. A well-controlled capture setup with a modest model reliably beats an excellent model on inconsistent images, and benchmark positions are measured on datasets nothing like yours.

Q Why are you raising licensing? That is a legal issue.

It becomes an engineering issue when it forces a rewrite. Several widely-used vision frameworks and pretrained weights carry copyleft licences whose terms can extend to network-delivered services, which may require a commercial licence for internal deployment. Others restrict weights to non-commercial research. Establishing the position now costs an email. Establishing it after the model is embedded in a production line costs a replacement programme. Add it to your model selection checklist alongside accuracy and latency.

Q Should we augment our training data?

Only along axes where the variation genuinely occurs. Brightness within the range your shifts actually produce, yes. Small rotations if part placement varies, yes. Ninety-degree rotation with a fixed camera and a fixed fixture, no — that teaches the model to ignore orientation when orientation may be diagnostic, and it costs accuracy. The test is simple: could this augmented image plausibly have come out of your camera? If not, you are teaching an invariance that does not exist.

Q Our model worked well in testing and degraded after deployment. Why?

Almost always one of four physical changes: lighting, camera or optics, part presentation, or product mix. A lamp aged, a camera was replaced with a slightly different sensor, a fixture wore, or a new variant entered production. None of these is

logged as a change to the AI system, so the degradation appears mysterious. The fix is cheap: keep a change log linking physical process changes to model monitoring, and check it first whenever performance moves.

B. Labels and Error Analysis

Q Our inspectors are experienced. Why would their labels be inconsistent?

Because experienced people apply judgement, and judgement varies at the boundary. Nobody disagrees about an obvious defect or an obviously good part. The disagreement is concentrated exactly where the model needs the sharpest signal. This is not a criticism of your inspectors — it is a criticism of the class definition they were given. Measure it before assuming: two inspectors, 100 images, independently. The number will be informative either way.

Q What agreement rate should we expect?

For clear-cut defect classes on good images, 90% or above is achievable. For subjective surface finish judgements, 70 to 80% is common and honest. Below about 60%, the class definition needs work before any modelling continues, because you are asking the model to learn a boundary that does not consistently exist. Whatever the number is, it is roughly your model's ceiling — which makes it the first thing to establish, not the last.

Q You are saying our model failures might actually be labelling errors. How common is that?

In our experience of industrial defect datasets, ten to thirty per cent of what gets recorded as a model failure turns out to be a labelling error on inspection. That is why Lab 1 has you check the label on thirty failures — not to embarrass anyone, but because it completely changes what you fund next. A team reporting 78% accuracy when 20% of the errors are ground-truth errors is measuring the label noise as much as the model.

Q Two of our defect classes are constantly confused. How do we fix it?

First check whether your inspectors confuse them too. Take 50 images the model got wrong and have two inspectors label them independently. If the inspectors also disagree, the model is faithfully learning your inconsistency and more data will not help. You then have two honest options: write a sharper definition with visual boundary examples and re-label, or merge the classes. Merging is underused — ask what decision actually depends on telling them apart. Frequently the answer is none.

Q Merging classes feels like giving up.

It is giving up on a distinction that cannot be made reliably, which is different. If two trained inspectors cannot separate scuffs from scratches consistently, then carrying that distinction into your model guarantees a permanent ceiling and a permanently confusing confusion matrix. Merging them into one surface-defect class and reporting a clean number is more honest and more useful. You can always split them later if someone establishes a definition that holds.

Q Our defects are tiny. Is that solvable?

Establish the pixel count first — how many pixels does the smallest defect you must catch occupy? Below about twenty, the honest answer is usually optical rather than algorithmic: more magnification, more cameras, or a two-stage inspection. Tiling and higher input resolution help and cost latency. Multi-scale architectures help somewhat. But if the information is not in the image at sufficient resolution, no model recovers it, and months spent trying is a common and avoidable loss.

Q How do we know whether to fix the data, the capture or the model?

The Lab 1 sequence answers it. Review fifty false negatives and fifty false positives individually. Check per-class recall rather than the aggregate. Audit the labels on thirty failures. Group failures by lighting, camera, shift and variant. By the

end you will be able to say which of the four it is with evidence — and in most industrial datasets it is labels or capture, not the model.

C. MLOps

Q We are a small team. Do we need all this MLOps machinery?

You need the five questions answered, not the machinery. Can you reproduce a result, roll back, tell it degraded, ship a change safely, retrain reliably? For a small team, honest answers might be: a versioned dataset in object storage, a spreadsheet of experiments with hypotheses recorded, a model registry that is a folder with a manifest, and a scheduled evaluation script. That is a week of work and it answers four of the five. Buying a platform to solve a discipline problem gives you an expensive platform and the same problem.

Q Which MLOps capability should we build first?

Versioning bound together — dataset, features, model, label guideline — because everything else depends on it and because retrofitting it is painful. Second, the evaluation gate, because it is what stops quality drifting downward one small exception at a time. Third, monitoring on the signals available without labels. Rollback and retraining automation can wait; those two cannot.

Q How long should rollback take?

Minutes, and the fact that most teams cannot answer this is the finding. Ask it as a concrete test in Lab 3: from noticing a problem to the previous model serving again, how long, and who does it? If nobody knows, that is the first gap to close, because the alternative during a real incident is a degraded model serving for days while someone works out the deployment path.

Q What is the evaluation gate actually checking?

Three things, all set before the run. The primary metric must exceed the current production model. No slice may regress by more than a stated margin — that is what stops an improvement on common classes hiding a regression on a rare critical one. And every regression case in the suite must pass. A model failing any of the three does not deploy and does not get argued into production. The discipline is identical to Day 1's pass-criterion-in-advance; the gate just enforces it organisationally.

Q Should we deploy with canary or shadow?

Shadow first, always, and for industrial vision it is unusually valuable. The new model runs on live images, its output is not used, and you compare its decisions against the current model and against inspector judgement on the same physical parts. You get real behaviour, latency and cost with zero risk. Canary after that, when you want real outcomes on a limited population.

Q How often should we retrain?

On triggers set in advance, with a scheduled cadence as a floor. Triggers: measured accuracy on a fresh labelled sample falling below the error budget; a defined volume of new labelled data accumulating; or a logged physical process change. What you should not do is retrain because one week looked bad — that is retraining on noise, and it introduces variance you then have to diagnose.

Q How do we know the model degraded if we do not have labels in production?

Four signals available without labels, and they are the early ones. Input distribution shift, which is the earliest of all. Confidence distribution shift. Rising deferral or escalation rate. And rising reviewer override rate, if your review decisions are captured. None tells you accuracy dropped; all tell you to go and check. Then you label a small fresh sample and find out. That combination is far cheaper than labelling continuously.

D. Active Learning and Feedback Loops

Q Is active learning worth the complexity?

When expert labelling time is scarce and production generates far more images than anyone can label — which describes almost every industrial CV programme — yes, clearly. The same labelling budget spent on uncertain cases rather than random ones produces substantially more improvement. And the simplest version is not complex: rank production images by model confidence, take the least confident thousand, send those for labelling. That is an afternoon of work.

Q Which sampling strategy should we start with?

Uncertainty sampling, because it is simplest — with one precondition. Check your model is calibrated first. If it is over-confident, which most neural networks are, it will select the wrong examples and the loop will underperform for a reason nobody diagnoses. Once a review loop is running, error-based sampling is stronger still, because cases where a human overturned the model are the highest-information examples you have.

Q Our reviewers already check parts. Is that not already a feedback loop?

Only if the decision is captured in a form that can reach training. In most operations the reviewer makes a call, the part is accepted or scrapped, and nothing about the model's proposal or the human's disagreement is stored. That is the single most common gap on this day — the most valuable data the system generates is discarded daily. Ask specifically: does your review decision get stored somewhere it can reach training? It is usually fixable in a week.

Q Will our reviewers actually correct labels, or just override?

Whichever the interface makes easier. If correcting takes four clicks and overriding takes one, you will get overrides and no training data. Design for correction as the default path. And close the loop visibly — tell reviewers when their corrections improved the model. Without that, correction rates decay within weeks. It is a human factor and it is more decisive than any technical choice in the loop.

Q Can we just retrain on everything the reviewers corrected?

Not without a gate and not without an audit. Corrections are labels and can be wrong — a reviewer having a bad shift, or drifting from the guideline, produces bad training data that degrades the model quietly. Sample-audit corrections against the guideline before they enter training, and put the retrained model through the evaluation gate like any other. Also make sure new cases go to training or evaluation, never both, or you will inflate every future measurement.

E. Difficult Questions — Handle With Care

These challenge the premise of the day or put you on the spot. Answer honestly rather than defensively — that is what works with an engineering audience.

Q You have spent the day telling us the model is not the problem. Is that not convenient?

It would be, if it were an assertion. It is a testable claim and Lab 1 tests it. Audit the labels on thirty failures, group the rest by capture condition, and check per-class recall. If the finding is that labels are clean, capture is controlled, coverage is good and the model is still wrong, then the model is the constraint and you should invest there. That happens. It is just less often than teams assume, and the diagnosis costs an afternoon either way.

Q Our vendor supplies the vision system. We cannot change the labels or the capture.

Then you can still do three things, and they are worth doing. Measure honestly on your own held-out data so you know the real number rather than the quoted one. Run the error analysis so you can tell the vendor specifically what fails and under which conditions — that conversation goes very differently with counts and slices than with "it does not work well." And tune the operating threshold, if you have access to it, because that is frequently the largest available gain and requires nothing from the vendor.

Q Improving labels means telling our quality team their inspection is inconsistent.

Frame it as a definition problem rather than a people problem, because that is what it usually is. The measurement is of the class definition, not of the inspectors — two skilled people given an ambiguous boundary will disagree, and that is expected. The output is a written standard with boundary samples, which is something quality functions already understand and generally welcome. Presented as "we need to tighten the standard", it is a collaboration. Presented as "your labels are wrong", it is a fight you will lose.

Q This is a lot of process for a model that might be replaced in a year.

Most of it outlives the model. The labelled dataset, the labelling guideline, the evaluation set, the failure taxonomy and the feedback loop are all model-independent assets — swap the architecture and they still apply. That is rather the argument for building them: they are the parts that compound, whereas the model itself is the part with the shortest half-life. The versioning and deployment machinery is genuinely model-specific, and that is the part to keep proportionate.

Q Our management wants a number for how much this will improve accuracy.

Give ranges tied to the diagnosis rather than a single figure. If the label audit shows 20% of errors are ground-truth errors, fixing labels recovers a substantial part of that and you can say so with the evidence attached. If threshold tuning shows a 30% error-cost reduction on the cost curve, that is a measured claim. What you should not do is promise a percentage before the diagnosis, because you would be guessing — and a promised number you miss costs more credibility than a range you meet.

Q Half the room is not working on computer vision. Is today relevant to them?

Modules 3 and 4 apply to every model-based system regardless of input type — versioning, evaluation gates, deployment, monitoring, retraining triggers and the feedback loop are the same for a maintenance model as for a vision one. Modules 1 and 2 are vision-specific in their examples and general in their method: the error taxonomy, the label-quality argument and the threshold work transfer directly. Ask the non-vision teams to translate each failure category into their own domain as we go; it works well.

General principle for the hard questions. The day's argument is falsifiable and the labs falsify it. Say that plainly. An engineering audience trusts a claim more when you tell them how to check it than when you defend it.

F. Quick Reference — One-Line Answers

If asked	Say
How many images do we need?	Depends which constraint you are hitting. Diagnose first.
Very few defect examples?	Anomaly detection on good parts, not supervised classification.
Detection or segmentation?	Segmentation only when extent or area drives a decision.
Which architecture?	Latency, pixels on target and licence constrain it more than accuracy.
Should we augment?	Only along axes where the variation genuinely occurs on your line.
Model degraded after deployment?	Check lighting, camera, presentation and product mix first.
What agreement rate to expect?	Whatever it is, it is roughly your ceiling. Measure it first.
Classes constantly confused?	Check whether your inspectors confuse them too.
Defects too small?	Count the pixels. Under twenty, the fix is optical.
Do we need MLOps tooling?	You need the five questions answered. Tooling is optional.
What does the gate check?	Primary metric up, no slice regressed, every regression case passing.
How long should rollback take?	Minutes. If nobody knows, that is the first gap.
Degradation without labels?	Input distribution, confidence, deferral rate, override rate.
Which sampling strategy?	Uncertainty sampling — after you have checked calibration.
Is our review a feedback loop?	Only if the decision is stored where training can reach it.

Companion documents: **Day 4 Presentation** (39 slides with speaker notes) and **Day 4 Study Material** (concept explanations with analogies and technical depth).